

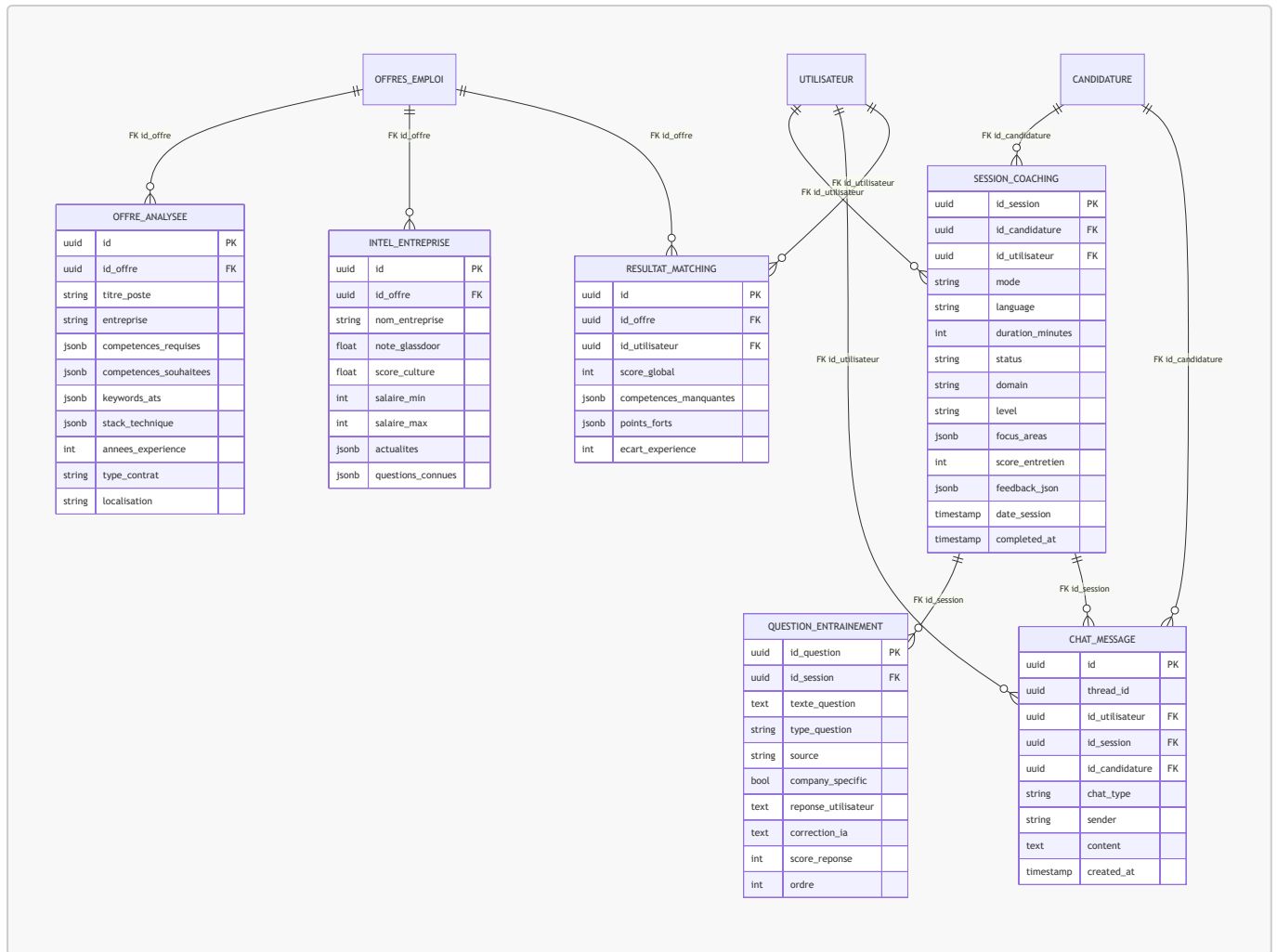
Ce document se base sur le backend .NET actuel et distingue deux couches de donnees :

1. le schema principal pilote par `AppDbContext`
2. les tables annexes partagees avec le module chatbot / les agents Python

- plusieurs relations vers `utilisateur` sont logiques au niveau applicatif mais ne sont pas forcées par une cle étrangère SQL dans le schéma principal
- `cv_history.template_slug -> cv_template.slug` est aussi une relation logique, pas une FK SQL
- `sourced_offer.promoted_offer_id -> offres_emploi.id` est une relation logique, pas une FK SQL
- le module chatbot garde encore une référence legacy à `public.offre` dans `ArenaService`, alors que le cœur backend persiste les offres dans `public.offres_emploi`

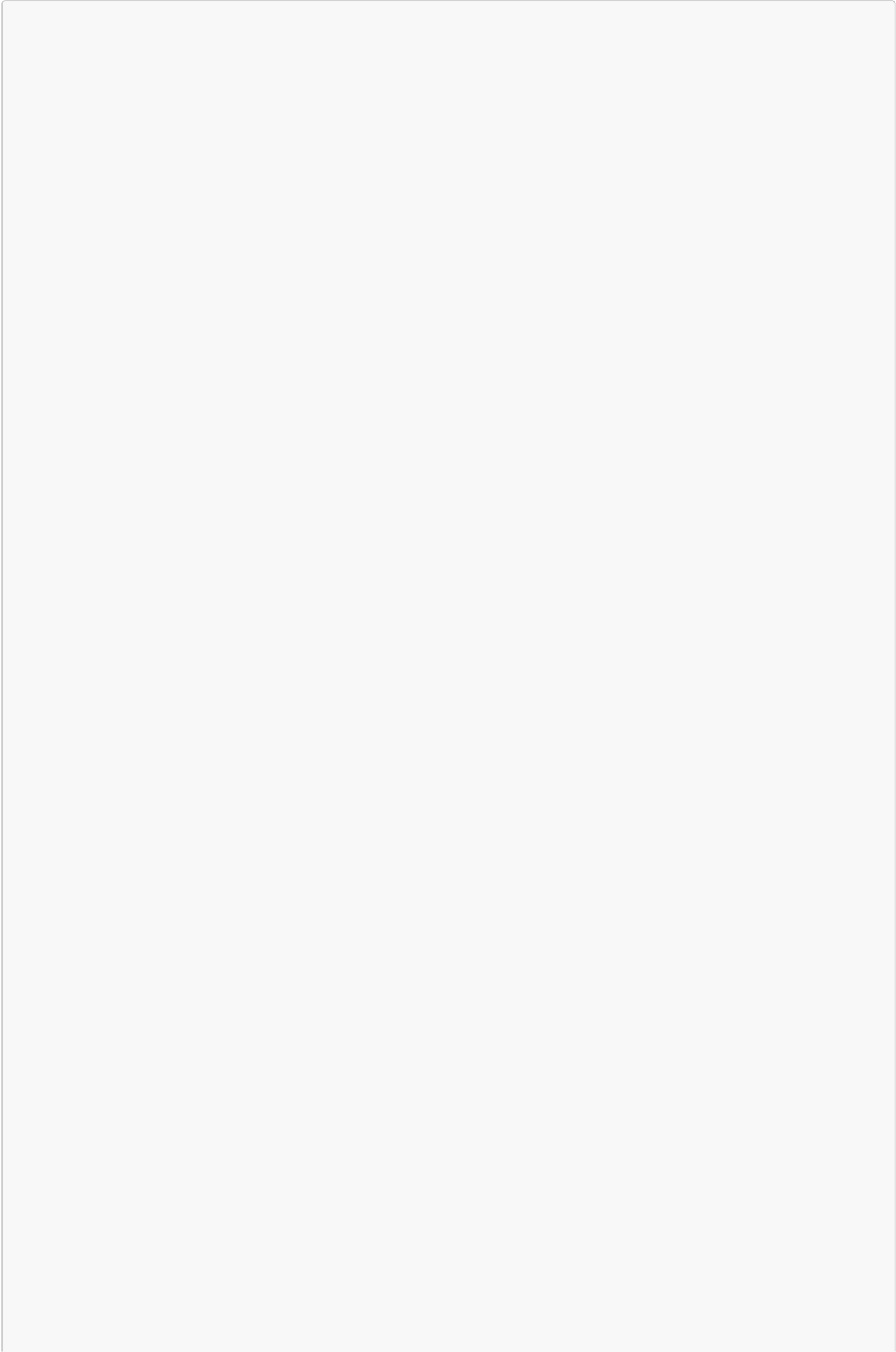
[illegible]

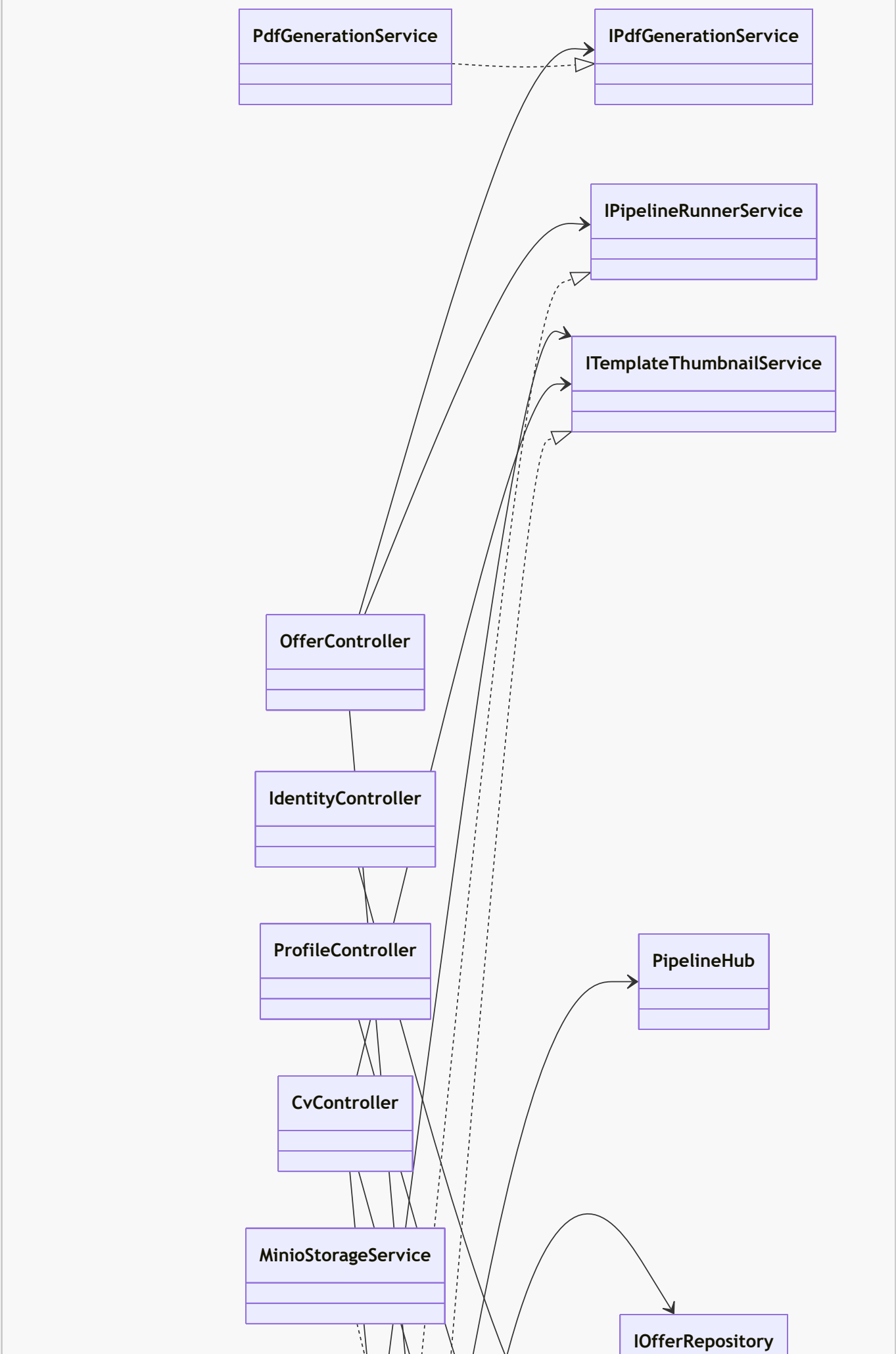
Ces tables existent dans les modeles ou dans `postgres/init.sql` et sont utilisees par `ArenaService` et les agents Python.

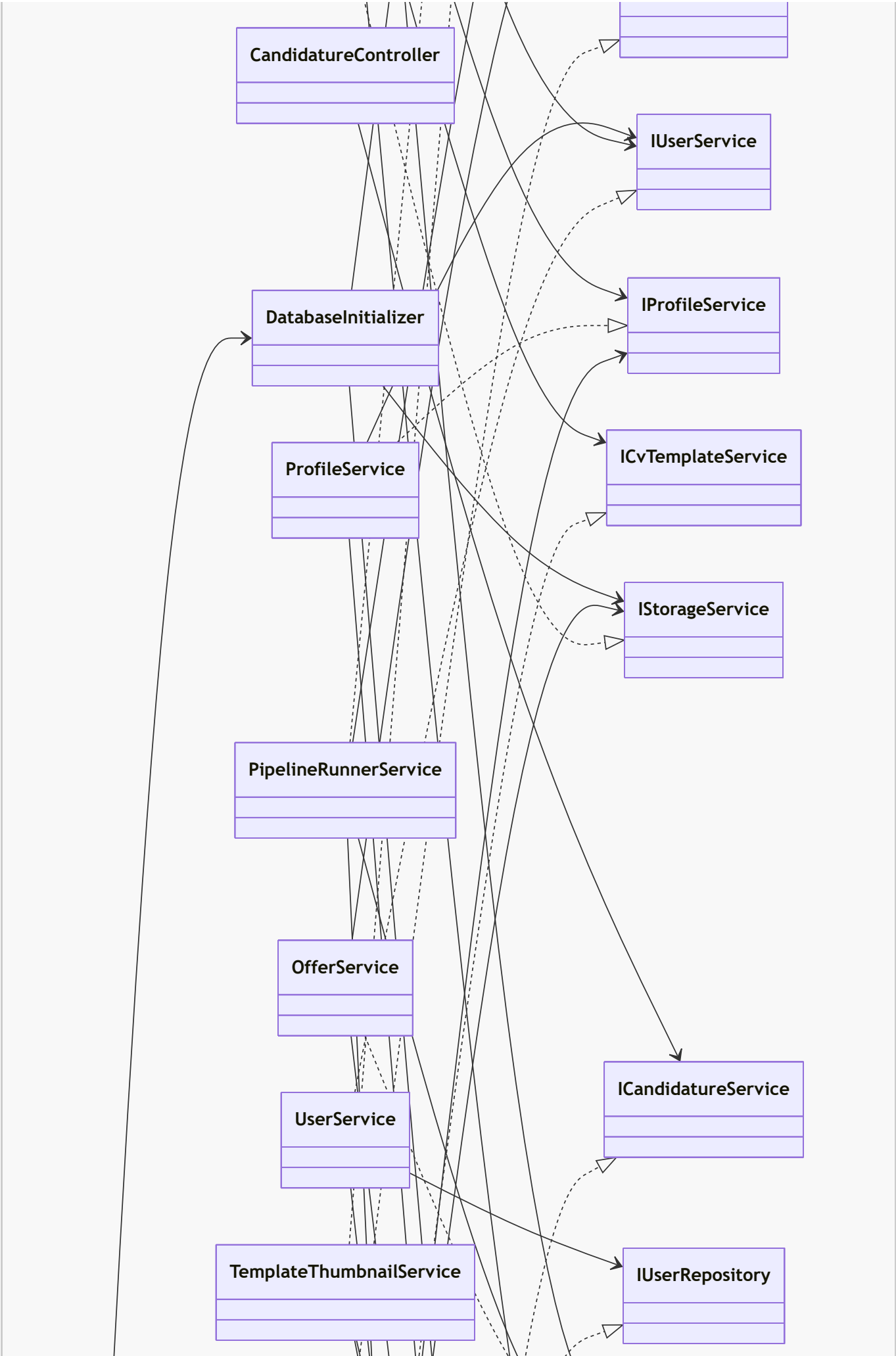


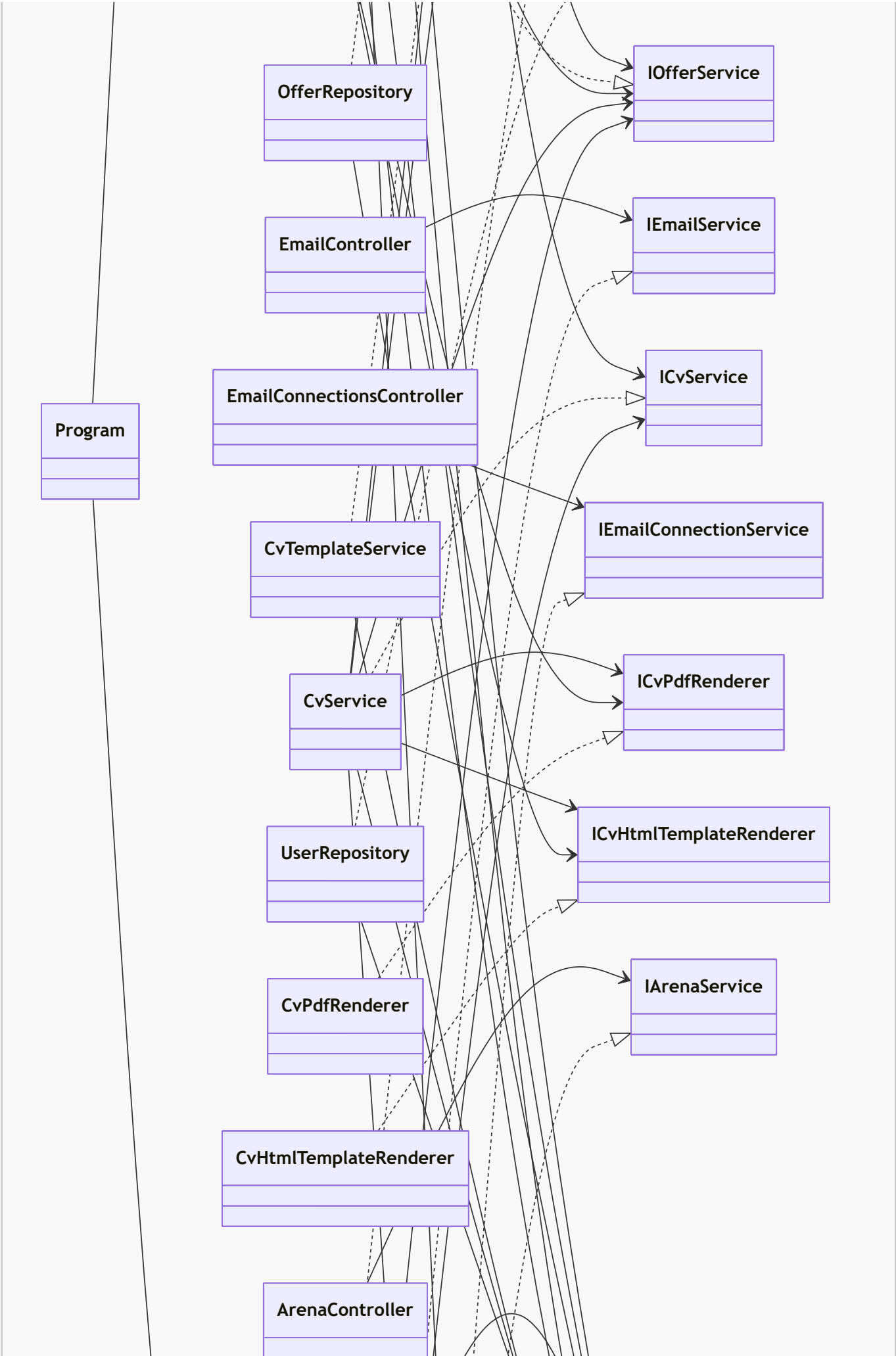
3. Backend class diagram

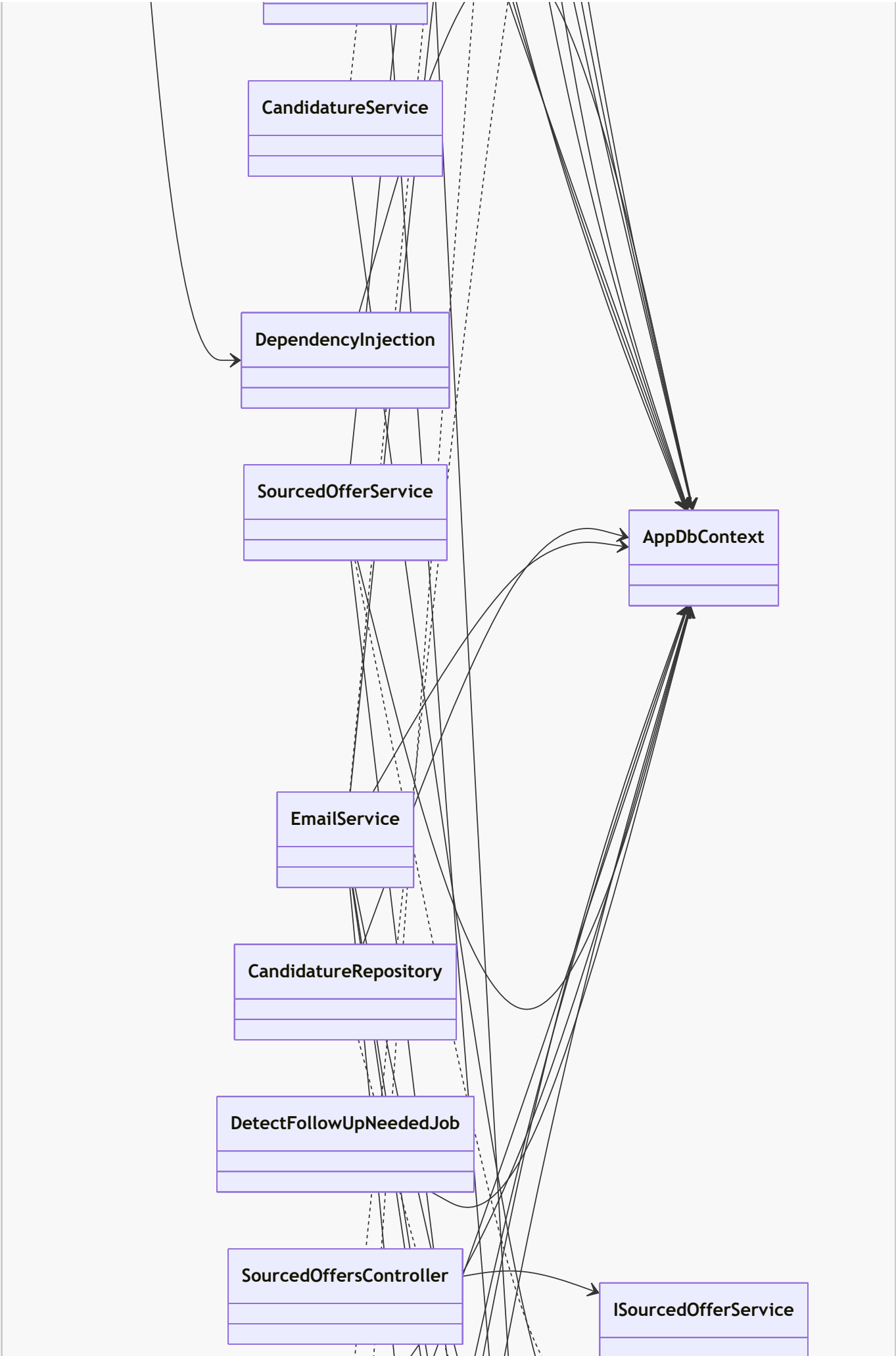
Le diagramme ci-dessous est volontairement architectural : controllers, services, repositories, infra et integrations. Les entites metier sont deja decrites dans les ER diagrams.

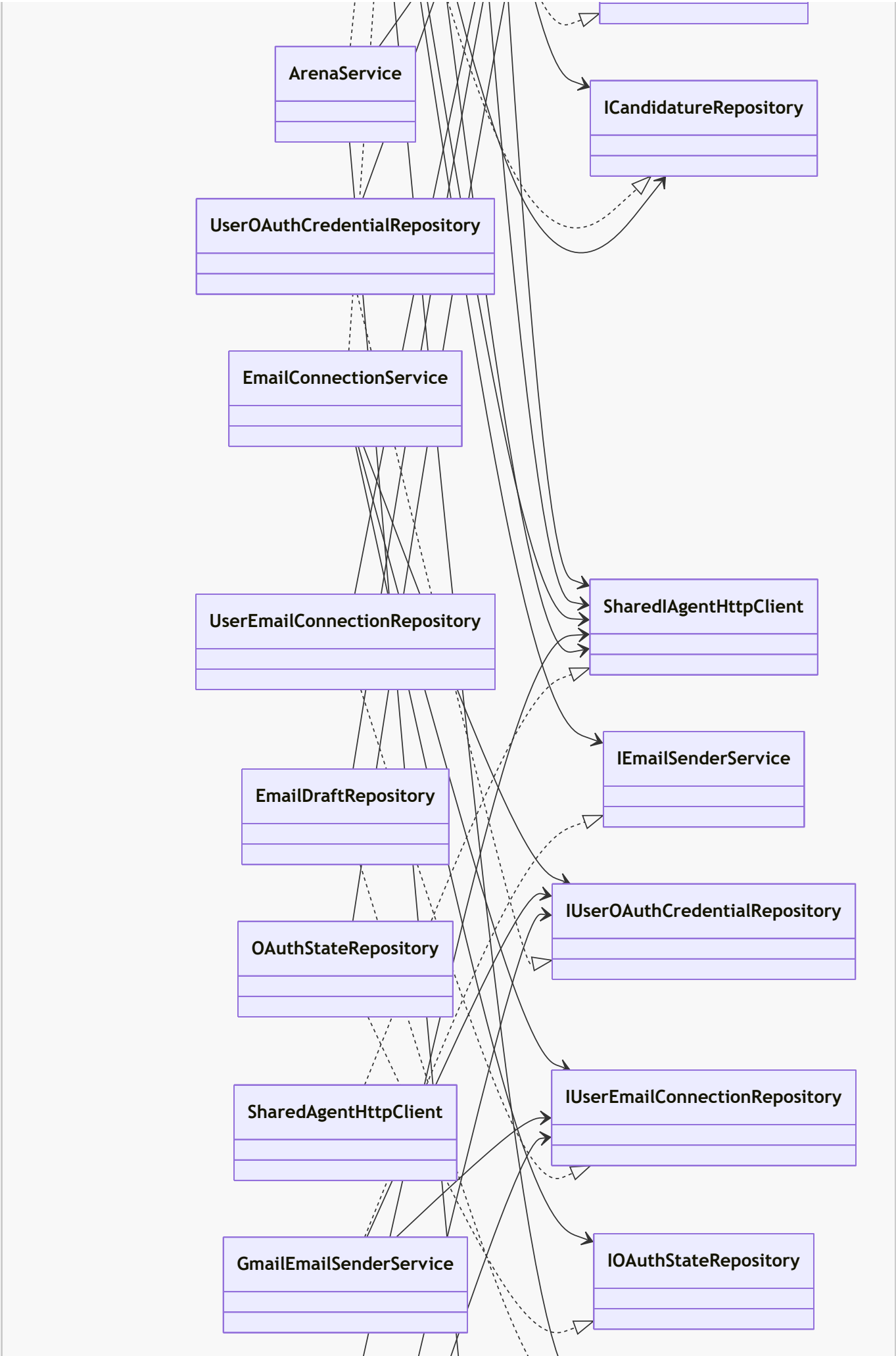


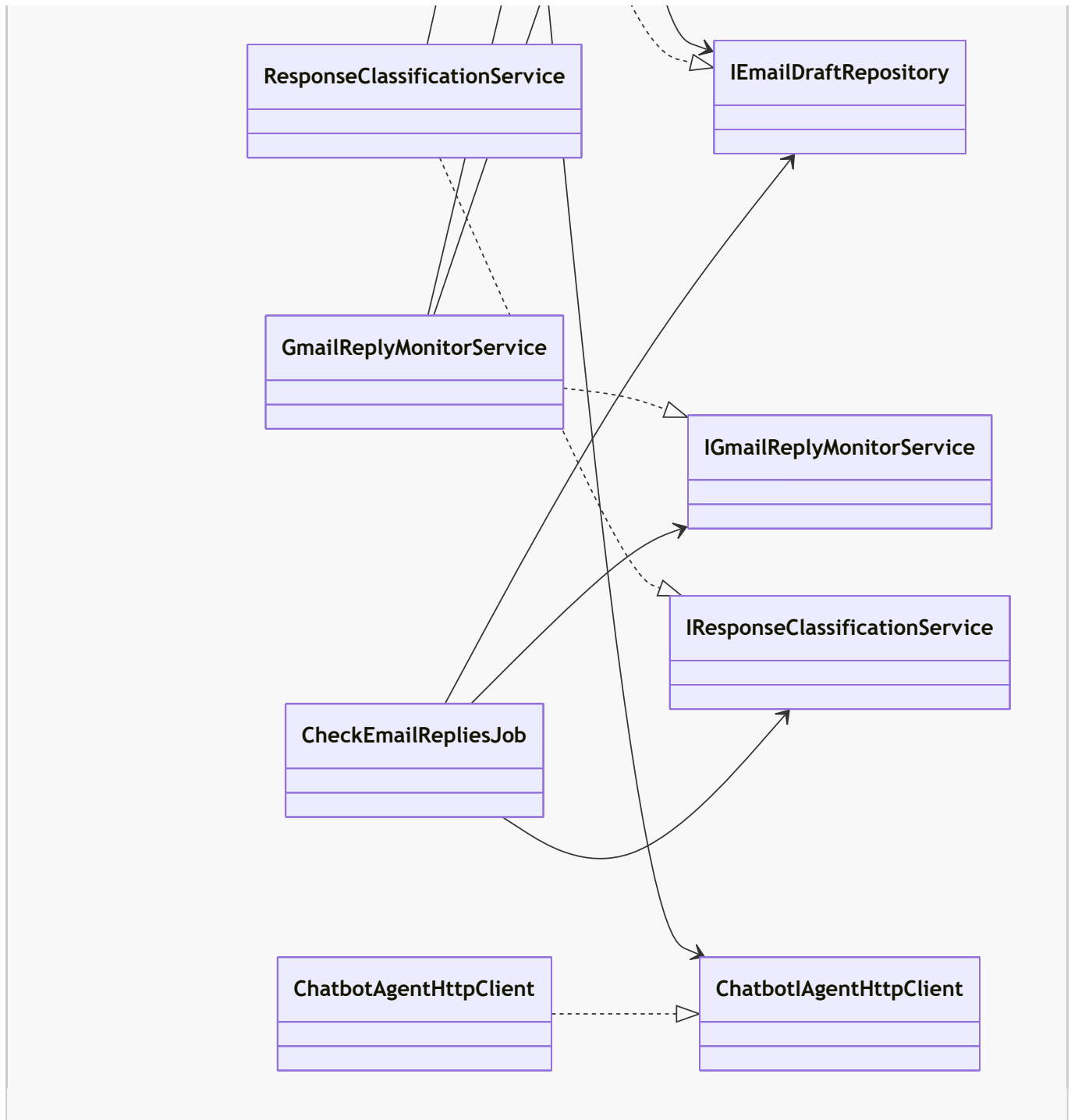












4. Source map

- `backend/data/AppDbContext.cs`
- `backend/data/DatabaseInitializer.cs`
- `backend/Modules/Offer/*`
- `backend/Modules/Profile/*`
- `backend/Modules/Cv/*`
- `backend/Modules/Email/*`
- `backend/Modules/Candidature/*`
- `backend/Modules/Identity/*`
- `backend/Modules/Sourcing/*`
- `backend/Modules/Chatbot/*`
- `backend/Jobs/*`

- `backend/SignalR/PipelineHub.cs`
- `postgres/init.sql`